

ПРАКТИЧЕСКАЯ РАБОТА
Исследование и ограничение методов протокола HTTP

Выполнил: Аньшин Сергей
Группа: ИСП-406

2026

1 Цель работы

Изучить методы протокола HTTP, определить методы, обрабатываемые веб-сервером, настроить явный запрет неиспользуемых методов и проверить, что после настройки основное приложение продолжает работать.

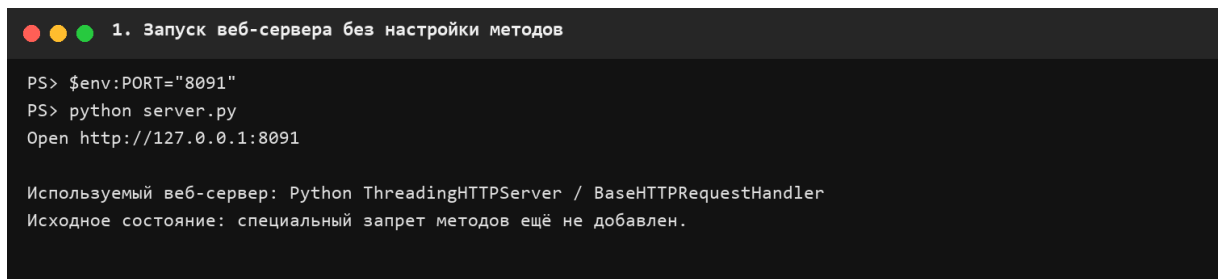
2 Задание

- поднять веб-сервер без специальной настройки методов;
- создать приложение, отправляющее запросы разными HTTP-методами;
- провести первичную разведку методов;
- настроить запрет неиспользуемых методов;
- проверить работу сайта;
- повторить разведку;
- сделать вывод о защите доступа по HTTP-методам.

3 Ход работы

3.1 Запуск веб-сервера без настройки методов

В качестве веб-сервера использован ThreadingHTTPServer из стандартной библиотеки Python с обработчиком BaseHTTPRequestHandler. Для первичного исследования была запущена версия сайта до добавления явного ограничения методов.



```
1. Запуск веб-сервера без настройки методов

PS> $env:PORT="8091"
PS> python server.py
Open http://127.0.0.1:8091

Используемый веб-сервер: Python ThreadingHTTPServer / BaseHTTPRequestHandler
Исходное состояние: специальный запрет методов ещё не добавлен.
```

Рисунок 1 — Запуск исходной версии веб-сервера

3.2 Приложение для отправки запросов

Создан файл http_methods_app.py. Приложение устанавливает соединение с сервером и отправляет запросы GET, POST, PUT и DELETE. Для каждого запроса выводятся статус, заголовок Allow и тело ответа.

```
2. Приложение отправляет запросы разными HTTP-методами

GET /
Статус: 200 OK
Allow: -
Ответ: <!doctype html>

<html lang="ru">

<head>

<meta charset="utf-8">

<meta name="viewport" content="width=device

POST /api/method-check
Статус: 404 Not Found
Allow: -
Ответ: {"error": "Маршрут не найден"}

PUT /api/method-check
Статус: 405 Method Not Allowed
Allow: GET, POST
Ответ: {"error": "Метод запрещён", "allowed": ["GET", "POST"]}

DELETE /api/method-check
Статус: 405 Method Not Allowed
Allow: GET, POST
Ответ: {"error": "Метод запрещён", "allowed": ["GET", "POST"]}
```

Рисунок 2 — Результат работы приложения с разными HTTP-методами

GET возвращает 200 OK. POST распознаётся сервером, но тестовый маршрут отсутствует, поэтому возвращается 404 Not Found. PUT и DELETE после настройки возвращают 405 Method Not Allowed и список разрешённых методов.

3.3 Первичная разведка методов

Для разведки создан файл `method_scanner.py`. Он последовательно отправляет GET, POST, HEAD, PUT, PATCH, DELETE, OPTIONS, TRACE и CONNECT.

```
3. Первичная разведка HTTP-методов

Разведка HTTP-методов: http://127.0.0.1:8091
METHOD    STATUS  ALLOW  РЕЗУЛЬТАТ
-----
GET       200    -      обрабатывается сервером
POST      404    -      обрабатывается сервером
HEAD      501    -      не реализован сервером
PUT       501    -      не реализован сервером
PATCH    501    -      не реализован сервером
DELETE    501    -      не реализован сервером
OPTIONS   501    -      не реализован сервером
TRACE     501    -      не реализован сервером
CONNECT   501    -      не реализован сервером
```

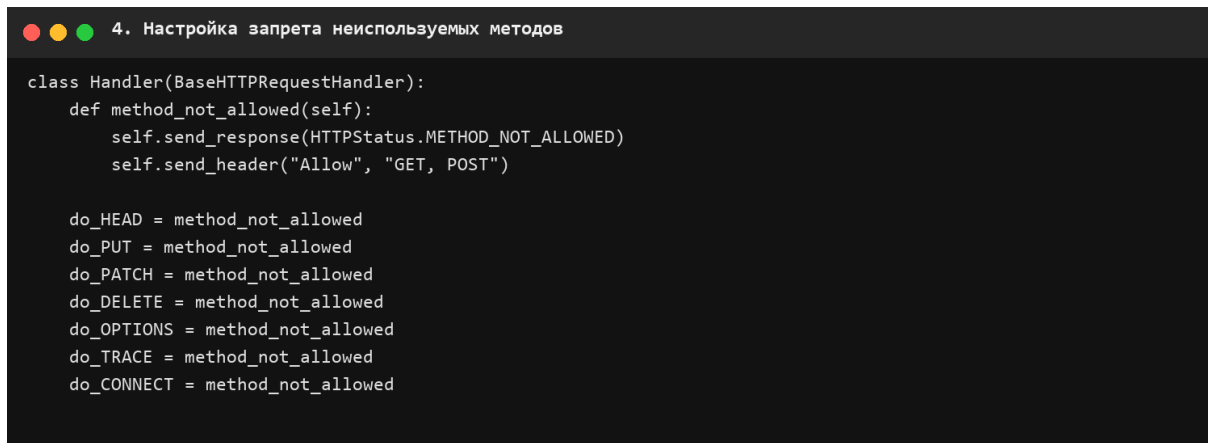
Рисунок 3 — Результаты первичной разведки HTTP-методов

До настройки GET и POST обрабатывались сервером. Код 404 у POST означает, что сам метод поддерживается, но указанного маршрута нет. Остальные методы возвращали 501 Not Implemented. Такой ответ сообщает

только об отсутствии обработчика и не является сформулированной политикой разрешённых методов.

3.4 Запрет неиспользуемых методов

Для сайта необходимы только GET для получения страниц и данных и POST для регистрации, входа, выхода, создания заявок и изменения статуса. Для остальных методов добавлен единый обработчик `method_not_allowed`.



```
4. Настройка запрета неиспользуемых методов

class Handler(BaseHTTPRequestHandler):
    def method_not_allowed(self):
        self.send_response(HTTPStatus.METHOD_NOT_ALLOWED)
        self.send_header("Allow", "GET, POST")

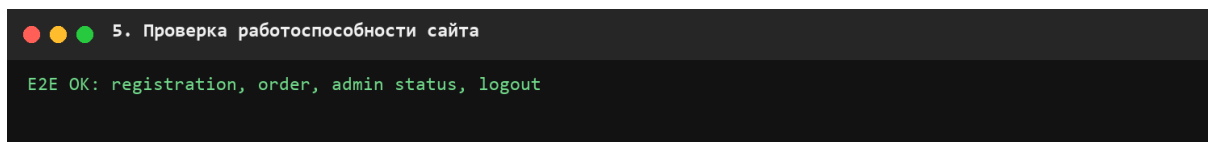
    do_HEAD = method_not_allowed
    do_PUT = method_not_allowed
    do_PATCH = method_not_allowed
    do_DELETE = method_not_allowed
    do_OPTIONS = method_not_allowed
    do_TRACE = method_not_allowed
    do_CONNECT = method_not_allowed
```

Рисунок 4 — Настройка запрета HTTP-методов

Запрещённые методы получают код 405 Method Not Allowed. Заголовок `Allow: GET, POST` сообщает клиенту разрешённый набор. Ответ формируется в JSON и получает общие заголовки безопасности сайта.

3.5 Проверка работоспособности приложения

После настройки выполнен сквозной автоматический тест: регистрация пользователя, создание заявки, вход администратора, изменение статуса заявки и выход из системы.



```
5. Проверка работоспособности сайта

E2E OK: registration, order, admin status, logout
```

Рисунок 5 — Успешное прохождение функционального теста

Сообщение E2E OK подтверждает, что запрет лишних HTTP-методов не нарушил основные функции сайта.

3.6 Повторная разведка

После настройки сканер повторно отправил тот же набор запросов.

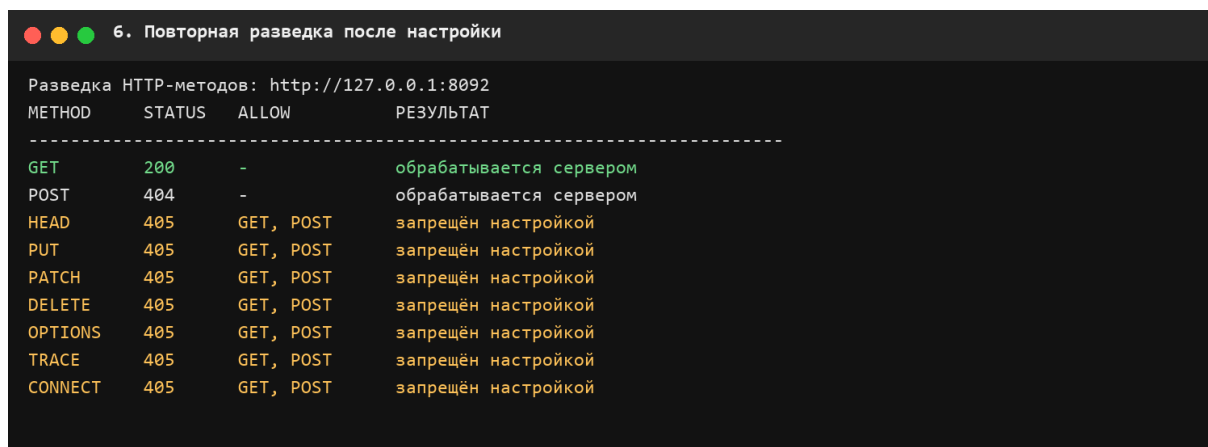


Рисунок 6 — Повторная разведка HTTP-методов

GET продолжает возвращать 200. POST распознаётся сервером. HEAD, PUT, PATCH, DELETE, OPTIONS, TRACE и CONNECT теперь явно запрещены кодом 405. Во всех ответах указан заголовок Allow: GET, POST.

4 Сравнение результатов

Метод	До настройки	После настройки	Результат
GET	200	200	разрешён и используется
POST	404 на тестовом маршруте	404 на тестовом маршруте	разрешён и используется
HEAD	501	405	явно запрещён
PUT	501	405	явно запрещён
PATCH	501	405	явно запрещён
DELETE	501	405	явно запрещён
OPTIONS	501	405	явно запрещён
TRACE	501	405	явно запрещён
CONNECT	501	405	явно запрещён

5 Вывод

В ходе работы исследованы основные методы HTTP. До настройки сервер обрабатывал GET и POST, а для остальных методов не имел обработчиков и возвращал 501. После настройки введена явная политика разрешённых методов: только GET и POST. Остальные методы возвращают 405 и заголовок Allow.

Ограничение методов уменьшает поверхность атаки: клиент не может применять PUT, PATCH или DELETE для непредусмотренного изменения ресурсов, TRACE не может использоваться для диагностического отражения запроса, CONNECT не может применяться как туннель, а OPTIONS не раскрывает дополнительные возможности. Одного ограничения методов недостаточно для полной защиты, поэтому оно применяется вместе с авторизацией, проверкой ролей, валидацией данных, безопасными cookie и заголовками безопасности.